Advanced Java OOP MCQs

please give the most extreme hard 20 mcqs based on the java oop all content..	2
other questions.	6
yes.	11
please give the another 20 extreme hard questions with explanation..	14
other questions.	20
yes.	26
give the another 20 mcqs..	32

please give the most extreme hard 20 mcqs based on the java oop all content.

Here's a set of **20 extremely challenging multiple-choice questions (MCQs)** covering various aspects of **Java Object-Oriented Programming (OOP)** concepts:

---

1. What happens when a constructor is declared as `private` in Java?

A. It cannot be instantiated in any class.
B. It can only be instantiated within its own class.
C. It can only be instantiated within its package.
D. It cannot be overloaded.

Answer: B

---

2. Which of the following is true about polymorphism in Java?

A. It is achieved using method overloading and overriding.
B. It is implemented only during runtime.
C. It allows methods in subclasses to have a different name than the parent class.
D. It enables Java to support multiple inheritance.

Answer: A

---

3. Which modifier is used to prevent method overriding?

A. private
B. final
C. static
D. protected

Answer: B

---

4. What will be the output of the following code?

```java


class A {
    public static void method1() {
        System.out.println("Static Method in Class A");
    }
}
class B extends A {
    public static void method1() {
        System.out.println("Static Method in Class B");
    }
}

public class Test {
```

```
        public static void main(String[] args) {
            A obj = new B();
            obj.method1();
        }
    }
```

A. Static Method in Class A
B. Static Method in Class B
C. Compilation Error
D. Runtime Error

Answer: A

---

5. Which of the following is NOT true about the `super` keyword?

A. It is used to call the parent class constructor.
B. It can be used to access a parent class's static methods.
C. It is used to access a parent class's methods or variables.
D. It must be the first statement in a constructor if used to call the parent constructor.

Answer: B

---

6. What does the `instanceof` operator return when applied between a null object and a class?

A. True
B. False
C. Compilation error
D. Runtime exception

Answer: B

---

7. Which of the following is valid for Java Interfaces?

A. Interfaces can have private methods.
B. Interfaces can have final variables.
C. Interfaces can have constructors.
D. Interfaces can have instance fields.

Answer: B

---

8. What will happen if a class implements two interfaces with identical method signatures but different return types?

A. Compilation error
B. Runtime error
C. Overloading occurs
D. The method will not be inherited

Answer: A

---

9. Consider the following code snippet. What will be the output?

```java
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;
}

public class Test {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.x);
    }
}
```

A. 10
B. 20
C. Compilation error
D. Runtime error

Answer: A

---

10. Which of these is true regarding abstract classes in Java?

A. Abstract classes cannot have a constructor.
B. Abstract classes can have final methods.
C. Abstract classes can be instantiated.
D. Abstract classes cannot extend another class.

Answer: B

---

11. Which of the following scenarios is best suited for the Singleton design pattern in Java?

A. Managing a database connection pool
B. Extending the functionality of a class
C. Performing inheritance
D. Allowing multiple instances of a class

Answer: A

---

12. What will be the result of attempting to serialize an object with a field marked `` `transient` ``?

A. The transient field will be serialized normally.
B. The transient field will not be serialized.
C. The transient field will throw an exception during serialization.
D. The transient field will be set to zero or null during serialization.

Answer: D

---

13. What is the correct way to implement method overriding with covariant return types?

A. The overridden method in the subclass must return a subtype of the return type in the superclass.
B. The overridden method in the subclass must return a supertype of the return type in the superclass.
C. The overridden method in the subclass must have a different name.
D. The overridden method must have private access in the superclass.

Answer: A

---

14. What will happen if you define a class without any constructors?

A. It results in a compilation error.
B. The JVM automatically provides a default no-argument constructor.
C. The class cannot be instantiated.
D. The class is abstract by default.

Answer: B

---

15. What is the difference between `throw` and `throws` in Java?

A. `throw` is used to declare exceptions, while `throws` is used to throw exceptions.
B. `throw` is used to throw exceptions, while `throws` is used to declare exceptions.
C. Both `throw` and `throws` are used interchangeably.
D. `throws` is used at runtime, while `throw` is used at compile time.

Answer: B

---

16. What happens when a class overrides the `equals()` method but does not override the `hashCode()` method?

A. The program will not compile.
B. Objects will always be equal.
C. Objects that are equal according to `equals()` may not be treated as equal by hash-based collections.
D. There will be no impact on the program behavior.

Answer: C

---

17. Can an abstract class have a static method in Java?

A. Yes, but it must also be declared abstract.
B. No, abstract classes cannot have static methods.
C. Yes, static methods are allowed in abstract classes.
D. It depends on the JVM.

Answer: C

---

18. What will happen if the `finalize()` method throws an exception?

A. The program will terminate.
B. The exception will be ignored, and the object will be garbage collected.
C. The object will not be garbage collected.
D. The exception will be propagated to the calling method.

Answer: B

---

19. Can an interface inherit another interface in Java? If yes, which type of inheritance is it?

A. Yes, it is single inheritance.
B. No, interfaces cannot inherit other interfaces.
C. Yes, it is multiple inheritance.
D. Yes, it is interface extension.

Answer: D

---

20. Which method of `Object` class can be overridden to customize the output of `System.out.println()` when printing an object?

A. finalize()
B. hashCode()
C. equals()
D. toString()

Answer: D

These questions are designed to test advanced knowledge of **Java OOP concepts**, including **inheritance, polymorphism, abstraction, encapsulation, interfaces**, and practical Java implementation nuances. Let me know if you'd like detailed explanations for any of these!



other questions



Here's another set of **20 advanced Java OOP MCQs** with different challenges to broaden the scope:

---

1. Which of the following is true about an `enum` in Java?

A. `enum` can extend a class.
B. `enum` cannot have methods.
C. `enum` can implement an interface.
D. All `enum` values must be lowercase.

Answer: C

---

2. What will happen if a subclass does not override an abstract method from the parent abstract class?

A. Compilation error
B. Runtime error
C. The subclass itself becomes abstract.
D. The abstract method is automatically implemented.

Answer: C

---

3. What will happen when a class inherits two interfaces that have default methods with the same signature?

A. Compilation error
B. Runtime error
C. The subclass must override the conflicting method.
D. The JVM randomly picks one method implementation.

Answer: C

---

4. What is the purpose of the `@Override` annotation in Java?

A. To enforce method overloading.
B. To ensure that a method is overriding a method from a superclass or interface.
C. To make a method final.
D. To optimize the overridden method.

Answer: B

---

5. Which of these statements about Java's `Object` class is incorrect?

A. Every class in Java implicitly extends the `Object` class.
B. The `Object` class provides the `clone()` method.
C. The `Object` class is abstract.
D. The `Object` class provides the `toString()` method.

Answer: C

---

6. What happens if a `try` block does not have a matching `catch` block?

A. The program terminates immediately.
B. The exception is ignored.
C. The exception is propagated to the calling method.
D. It causes a compilation error.

Answer: C

---

7. Which of the following is true about method overriding and access modifiers?

A. A method in the subclass cannot have broader access than the overridden method in the superclass.
B. A method in the subclass must have the same access level as the overridden method in the superclass.
C. A method in the subclass can have narrower access than the overridden method in the superclass.
D. Access modifiers are not relevant in method overriding.

Answer: A

---

8. What will the following code output?

```
java


  class Test {
      public void display(Object obj) {
          System.out.println("Object");
      }

      public void display(String str) {
          System.out.println("String");
      }

      public static void main(String[] args) {
          Test test = new Test();
          test.display(null);
      }
  }
```

A. Object
B. String
C. Compilation error
D. Runtime error

Answer: B

---

9. Can a static nested class access non-static members of its outer class?

A. Yes, always.
B. No, never.
C. Only through an instance of the outer class.
D. Only when declared as `**final**`.

Answer: C

---

10. Which of these can be a part of an abstract class?

A. Static methods
B. Private constructors
C. Concrete methods
D. All of the above

Answer: D

---

11. What will the following code print?

```java
interface A {
    default void show() {
        System.out.println("Interface A");
    }
}

interface B {
    default void show() {
        System.out.println("Interface B");
    }
}

class Test implements A, B {
    public void show() {
        A.super.show();
    }

    public static void main(String[] args) {
        Test test = new Test();
        test.show();
    }
}
```

A. Interface A
B. Interface B
C. Compilation error
D. Runtime error

Answer: A

---

12. What does `super()` do in a constructor?

A. Calls another constructor in the same class.
B. Calls a method in the superclass.
C. Calls the parent class constructor.
D. Calls the default constructor of the same class.

Answer: C

---

13. Which of the following statements is true about constructor overloading in Java?

A. Constructors cannot be overloaded.
B. Overloaded constructors must have the same number of parameters.
C. Overloaded constructors must differ in parameter types, order, or number.
D. Constructors with different access modifiers cannot be overloaded.

Answer: C

---

14. What is the difference between `has-a` and `is-a` relationships in Java?

A. `has-a` refers to inheritance, while `is-a` refers to composition.
B. `is-a` refers to inheritance, while `has-a` refers to composition.

C. `is-a` is static, while `has-a` is dynamic.
D. `has-a` involves interfaces, while `is-a` involves classes.

Answer: B

---

15. Which statement is true about the `final` keyword in Java?

A. A final class cannot have constructors.
B. A final method can be overridden.
C. A final variable must be initialized when declared or in a constructor.
D. A final method can be abstract.

Answer: C

---

16. What happens when you call the `wait()` method in a synchronized block?

A. The thread releases the lock and enters the waiting state.
B. The thread remains in the synchronized block.
C. The thread moves to the runnable state.
D. The thread terminates immediately.

Answer: A

---

17. What will the following code output?

```java
class A {
    public void print() throws RuntimeException {
        System.out.println("Class A");
    }
}

class B extends A {
    public void print() throws ArithmeticException {
        System.out.println("Class B");
    }
}

public class Test {
    public static void main(String[] args) {
        A obj = new B();
        obj.print();
    }
}
```

A. Class A
B. Class B
C. Compilation error
D. Runtime error

Answer: B

---

18. What will happen if the following code is executed?

```java
class A {
    final int x = 10;

    public void changeX() {
        x = 20;
    }
}
```

A. `x` is changed to 20.
B. Compilation error.
C. Runtime error.
D. The value of `x` is 20, but only within `changeX()`.

Answer: B

---

19. Can you override a private method in Java?

A. Yes, always.
B. No, because private methods are not inherited.
C. Only if the subclass is in the same package.
D. Only if the private method is static.

Answer: B

---

20. Which of the following is true about the `finally` block in Java?

A. The `finally` block is executed only if an exception is thrown.
B. The `finally` block is always executed, even if no exception is thrown.
C. The `finally` block is executed only if `catch` is executed.
D. The `finally` block is never executed if an exception is thrown.

Answer: B

These questions dive into the depths of **inheritance, polymorphism, interfaces, exceptions, final modifiers, and multithreading** to challenge your knowledge of Java OOP concepts. Let me know if you'd like detailed explanations!

yes

Here are **detailed explanations** for the answers to the second set of Java OOP MCQs:

---

1. `enum` can implement an interface

- Enums in Java can implement interfaces to define behavior. However, they cannot extend classes because they already extend `java.lang.Enum`.
  Answer: C

---

2. Subclass without overriding abstract methods

- If a subclass doesn't implement all abstract methods of its parent class, it must itself be declared abstract.
  Answer: C

---

3. Conflicting default methods in interfaces

- When a class implements two interfaces with conflicting default methods, the subclass must resolve the conflict by overriding the method.
  Answer: C

---

4. Purpose of `@Override` annotation

- The `@Override` annotation ensures the method is correctly overriding a superclass or interface method. If it's not, the compiler throws an error.
  Answer: B

---

5. Incorrect statement about `Object` class

- The `Object` class is not abstract; it is concrete and can be directly instantiated. All other statements are correct.
  Answer: C

---

6. Behavior of `try` block without a matching `catch` block

- If no `catch` block matches the thrown exception, the exception propagates to the calling method, potentially terminating the program.
  Answer: C

---

7. Method overriding and access modifiers

- In method overriding, the overriding method cannot have a more restrictive access modifier than the overridden method.
  Answer: A

---

8. Overloaded method resolution

- `null` matches the most specific method. Since `String` is more specific than `Object`, `test.display(null)` calls the `display(String)` method.
  Answer: B

---

9. Static nested class accessing outer class members

- A static nested class can only access non-static members of its enclosing class through an instance of the outer class.
  Answer: C

---

10. Abstract class capabilities

- Abstract classes can contain static methods, concrete methods, and private constructors. They can even serve as a base for utility patterns.
  Answer: D

---

11. Resolving default method conflict

- The `A.super.show()` explicitly resolves the conflict between the default methods of `A` and `B`, resulting in the execution of `Interface A`.
  Answer: A

---

12. Purpose of `super()`

- The `super()` keyword is used to call the parent class constructor, which must be the first statement in a constructor if used.
  Answer: C

---

13. Constructor overloading

- Overloaded constructors must differ in parameter types, number, or order. Access modifiers and return types do not affect overloading.
  Answer: C

---

14. Difference between `has-a` and `is-a` relationships

- A `has-a` relationship refers to composition (e.g., a class having fields of another class type), while an `is-a` relationship refers to inheritance.
  Answer: B

---

15. `final` keyword and variables

- Final variables must be initialized when declared or within a constructor. They cannot be modified afterward.
  Answer: C

---

16. Behavior of `wait()` in a synchronized block

- The `wait()` method releases the lock and pauses the thread execution until `notify()` or `notifyAll()` is called on the same monitor.

Answer: A

---

17. Overridden method throwing a subclass exception

- The overridden method in `B` can throw a more specific exception (ArithmeticException) than the exception declared in the superclass (RuntimeException). The output is "Class B".
  Answer: B

---

18. Attempt to modify a `final` variable

- A final variable cannot be reassigned after being initialized. Attempting to modify it causes a compilation error.
  Answer: B

---

19. Overriding private methods

- Private methods are not inherited, so they cannot be overridden. They are class-specific.
  Answer: B

---

20. Behavior of the `finally` block

- The `finally` block is always executed after the `try` or `catch` block, regardless of whether an exception is thrown or caught.
  Answer: B

If you need more detailed examples for any specific question, let me know!

please give the another 20 extreme hard questions with explanation.

Here is another set of **20 extreme hard Java OOP MCQs** with **detailed explanations**:

---

1. Which of the following statements about `final` classes is true?

A. A `final` class can be extended.
B. A `final` class cannot implement interfaces.
C. A `final` class can have `final` methods.
D. A `final` class cannot have static methods.

Answer: C
Explanation:
A `final` class cannot be extended but can implement interfaces. It can contain `final` methods, static methods, and instance variables.

---

2. What happens when you explicitly define a parameterized constructor but do not define a default constructor?

A. The compiler automatically provides a default constructor.
B. The program will throw an error if the default constructor is used.
C. The default constructor is still available for use.
D. It creates a private default constructor.

Answer: B
Explanation:
If you define any constructor explicitly, the compiler does not generate a default constructor. Attempting to call the default constructor will result in a compilation error.

---

3. Can we override a `static` method in Java?

A. Yes, always.
B. No, static methods are hidden, not overridden.
C. Only if the method is `protected`.
D. Only if it is in a `final` class.

Answer: B
Explanation:
Static methods are associated with the class, not the object. You can hide them by redefining them in the subclass, but it is not considered overriding.

---

4. Which of the following is true about `transient` variables?

A. They are serialized by default.
B. They cannot be initialized.
C. They are skipped during serialization.
D. They can only be declared in `static` blocks.

Answer: C
Explanation:
A `transient` variable is not serialized when an object is serialized, meaning its value will not persist across object streams.

---

5. What is the output of the following code?

```java
class A {
    static int i = 10;
    static {
        i = i-- - --i;
    }
}

class B extends A {
    static {
        i = i++ + ++i;
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        System.out.println(B.i);
    }
}
```

A. 8
B. 9
C. 10
D. Compilation error

Answer: A
Explanation:

- First static block in `A`: `i = 10 - 8 = 2`.
- Second static block in `B`: `i = 2 + 6 = 8`.

---

6. Can an abstract class have a constructor?

A. Yes, but it cannot be called.
B. Yes, and it is called when an object of a subclass is created.
C. No, abstract classes cannot have constructors.
D. Only if all methods are implemented.

Answer: B
Explanation:
Abstract classes can have constructors, and these are called when a subclass object is created to initialize the abstract class's part of the object.

---

7. What happens if you make a method `synchronized`?

A. It cannot throw an exception.
B. It can only be called by one thread at a time.
C. It will execute faster.
D. It can only be called in a `try` block.

Answer: B
Explanation:
A synchronized method ensures that only one thread can execute the method at a time, locking the object or class.

---

8. Which of the following is true about `instanceof` in Java?

A. It checks if an object is an instance of a class.
B. It checks if a class is a subclass of another class.
C. It compares the memory addresses of two objects.
D. It only works with interfaces.

Answer: A
Explanation:

The `instanceof` operator checks whether an object is an instance of a specified class or implements an interface.

---

9. What is the result of calling `hashCode()` on two equal objects?

A. It must return different values.
B. It must return the same value.
C. It depends on the `equals()` method implementation.
D. It depends on the JVM.

Answer: B
Explanation:
If two objects are considered equal according to the `equals()` method, they must have the same `hashCode()` value.

---

10. What happens if a `finally` block contains a `return` statement?

A. The return in `finally` overrides any return in `try` or `catch`.
B. It causes a compilation error.
C. The return in `finally` is ignored.
D. It behaves the same as `try` or `catch`.

Answer: A
Explanation:
A `return` statement in `finally` overrides any previous return values from `try` or `catch`.

---

11. Can we declare an interface method as `private`?

A. Yes, always.
B. No, interfaces cannot have private methods.
C. Yes, but only with default methods.
D. Yes, from Java 9 onwards.

Answer: D
Explanation:
From Java 9 onwards, interfaces can have `private` helper methods.

---

12. What will happen if we do not provide a custom `equals()` method in a class?

A. The class cannot be compared with other classes.
B. The `==` operator will be used instead.
C. The default implementation in `Object` will be used.
D. A compilation error will occur.

Answer: C
Explanation:
The default implementation of `equals()` in `Object` compares memory references.

---

13. What is the output of the following code?

```java
class Test {
    public static void main(String[] args) {
        Integer x = 128, y = 128;
        System.out.println(x == y);
    }
}
```

A. true
B. false
C. Compilation error
D. Runtime error

Answer: B
Explanation:
For values greater than 127, `Integer` objects are not cached, so `x` and `y` point to different objects.

---

14. Which of the following features are allowed in a `sealed` class?

A. It must be `abstract`.
B. It restricts which classes can inherit it.
C. It must define a `final` method.
D. It cannot be subclassed.

Answer: B
Explanation:
`Sealed` classes, introduced in Java 15, allow controlled inheritance by specifying permitted subclasses.

---

15. Can a constructor be synchronized in Java?

A. Yes
B. No
C. Only in static contexts
D. Only in multi-threaded programs

Answer: B
Explanation:
Constructors cannot be synchronized because it makes no sense to lock an object before it is fully constructed.

---

16. What is the output of the following code?

```java
class Test {
    static int a = 1111;

    static {
        a = a-- - --a;
    }
```

```
        public static void main(String[] args) {
            System.out.println(a);
        }
    }
```

A. 0
B. 1
C. 2
D. -1

Answer: D
Explanation:
The expression modifies and evaluates `a` in steps: `1111 - 1109 = -1`.

---

17. Can a `record` class be serialized in Java?

A. Yes, always.
B. No, never.
C. Only if it implements `Serializable`.
D. Only if it overrides `toString()`.

Answer: C
Explanation:
Java `record` classes can be serialized if they implement the `Serializable` interface.

---

18. Can a Java `record` have mutable fields?

A. Yes
B. No
C. Only if they are declared `volatile`.
D. Only if they are declared `final`.

Answer: B
Explanation:
Fields in a `record` are inherently final and immutable.

---

19. Which feature is not part of the Java Reflection API?

A. Accessing private fields
B. Creating new classes dynamically
C. Invoking methods dynamically
D. Modifying the structure of classes

Answer: D
Explanation:
Java Reflection can inspect but cannot modify class structure dynamically.

---

20. What is the difference between `List<? extends T>` and `List<? super T>`?

A. `<? extends T>` allows reading elements as type `T` but restricts adding.
B. `<? super T>` allows adding elements of type `T` but restricts reading.
C. Both A and B.
D. None of the above.

Answer: C
Explanation:
`<? extends T>` is covariant, while `<? super T>` is contravariant.

If you want explanations for specific questions in more depth, let me know!

other questions

Here are **20 more extremely hard Java OOP MCQs** with explanations:

---

1. Which of the following is true for `protected` access modifier?

A. It is accessible only within the same class.
B. It is accessible only within the same package.
C. It is accessible in the same package and in subclasses outside the package.
D. It is accessible everywhere.

Answer: C
Explanation:
The `protected` modifier allows access within the same package and from subclasses, even if they are in different packages.

---

2. What is the output of the following code?

```java
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void display() {
        System.out.println(super.x + x);
    }
}

public class Test {
    public static void main(String[] args) {
        Child c = new Child();
        c.display();
    }
}
```

A. 10
B. 20
C. 30
D. Compilation error

Answer: C
Explanation:
The `super.x` refers to the parent class variable (`10`), and `x` refers to the child class variable (`20`), giving `10 + 20 = 30`.

---

3. Can a class implement two interfaces that contain methods with the same signature?

A. No, it causes ambiguity.
B. Yes, if the class provides its own implementation.
C. Only if the interfaces are from the same package.
D. Only if one of the methods is default.

Answer: B
Explanation:
A class can implement two interfaces with methods of the same signature, but it must provide its own implementation to resolve ambiguity.

---

4. Can a nested class be declared `abstract`?

A. No, nested classes cannot be abstract.
B. Yes, but it cannot have constructors.
C. Yes, and it can be instantiated only through its subclass.
D. Yes, but only if it is static.

Answer: C
Explanation:
An abstract nested class can be instantiated through its concrete subclass.

---

5. What happens when we serialize an object that contains a `transient` variable?

A. The transient variable's value is set to null or default value during deserialization.
B. The value is preserved.
C. Serialization fails.
D. The variable is marked as static.

Answer: A
Explanation:
During deserialization, the value of a `transient` variable is set to its default (null for objects, 0 for primitives).

---

6. What is the default access modifier for a method in an interface?

A. private
B. public
C. protected
D. default

Answer: B
Explanation:
Methods in an interface are implicitly `public` unless explicitly declared `private` (Java 9+).

---

7. What is the output of this code?

```java
class A {
    void method(Number n) {
        System.out.println("Number");
    }
}

class B extends A {
    void method(Double n) {
        System.out.println("Double");
    }
}

public class Test {
    public static void main(String[] args) {
        B b = new B();
        b.method(10);
    }
}
```

A. Number
B. Double
C. Compilation error
D. Runtime error

Answer: A
Explanation:
The `method(Number n)` in class `A` is inherited by `B`. Since `10` is an `Integer`, it matches the `Number` parameter.

---

8. Can you override a `private` method?

A. Yes
B. No
C. Only if the method is called by another class.
D. Only in the same package.

Answer: B
Explanation:
`private` methods are not inherited, so they cannot be overridden.

---

9. What happens when a thread calls `notify()` in a synchronized block?

A. It wakes up all waiting threads.
B. It wakes up one random thread waiting on the same monitor.
C. It terminates the current thread.
D. It releases the lock on the monitor.

Answer: B
Explanation:
The `notify()` method wakes up one thread waiting on the same monitor object.

---

10. What will be the output of the following code?

```java
class Test {
    public static void main(String[] args) {
        String s1 = "Test";
        String s2 = new String("Test");
        System.out.println(s1 == s2);
    }
}
```

A. true
B. false
C. Compilation error
D. Runtime error

Answer: B
Explanation:
The `==` operator compares memory references. `s1` refers to a string in the string pool, while `s2` is a separate object.

---

11. Can `super()` and `this()` appear in the same constructor?

A. Yes
B. No
C. Only if `super()` is called last.
D. Only if `this()` is static.

Answer: B
Explanation:
`super()` and `this()` cannot appear together in the same constructor because both must be the first statement.

---

12. What is the output of this code?

```java
class Parent {
    static void display() {
        System.out.println("Parent");
    }
```

```
}

class Child extends Parent {
    static void display() {
        System.out.println("Child");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent p = new Child();
        p.display();
    }
}
```

A. Parent
B. Child
C. Compilation error
D. Runtime error

Answer: A
Explanation:
Static methods are resolved at compile time based on the reference type, not the object type.

---

13. What is the result of attempting to access an overridden method in a superclass constructor?

A. The subclass method is called.
B. The superclass method is called.
C. Compilation error.
D. Runtime error.

Answer: B
Explanation:
During superclass constructor execution, overridden methods in the subclass are not called because the subclass is not yet initialized.

---

14. Can an interface extend another interface?

A. No
B. Yes, it can extend multiple interfaces.
C. Yes, but only one interface.
D. Yes, if it is `public`.

Answer: B
Explanation:
An interface can extend multiple interfaces, allowing multiple inheritance.

---

15. What happens when we override the `equals()` method but not `hashCode()`?

A. No effect.
B. Causes `HashMap` and `HashSet` to behave incorrectly.

C. Throws a compilation error.
D. Prevents object creation.

Answer: B
Explanation:
If `equals()` is overridden without `hashCode()`, collections like `HashMap` and `HashSet` may behave incorrectly.

---

16. What does the `volatile` keyword ensure?

A. It ensures that a variable is thread-safe.
B. It ensures changes to a variable are visible to all threads.
C. It prevents variables from being null.
D. It makes a variable static.

Answer: B
Explanation:
`volatile` ensures changes to a variable are immediately visible to all threads but does not make the variable inherently thread-safe.

---

17. What will happen if `finalize()` throws an exception?

A. The JVM rethrows the exception.
B. The program terminates.
C. The exception is ignored.
D. Compilation error.

Answer: C
Explanation:
If `finalize()` throws an exception, it is ignored by the JVM.

---

18. Can we override a `default` method in Java?

A. No
B. Yes
C. Only in abstract classes
D. Only in the same package

Answer: B
Explanation:
A `default` method from an interface can be overridden in a class or a subclass.

---

19. Which of the following is not part of Java's memory model?

A. Heap
B. Stack
C. Kernel memory
D. Method Area

Answer: C
Explanation:
The Java memory model includes Heap, Stack, Method Area, and others, but not Kernel memory.

---

20. Can a `static` block throw an exception?

A. Yes, any exception.
B. No
C. Only unchecked exceptions.
D. Only checked exceptions.

Answer: C
Explanation:
Static blocks can throw unchecked exceptions (runtime exceptions) but not checked exceptions.

If you'd like more such challenging questions, let me know!

yes

Here are **20 more extreme Java OOP MCQs** with detailed explanations:

---

1. What is the output of the following code?

```java
class A {
    public void display() throws Exception {
        System.out.println("A");
    }
}

class B extends A {
    public void display() {
        System.out.println("B");
    }
}

public class Test {
    public static void main(String[] args) {
        A obj = new B();
        try {
            obj.display();
        } catch (Exception e) {
            System.out.println("Exception");
        }
    }
}
```

A. A
B. B

C. Compilation error
D. Exception

Answer: B
Explanation:
In the overridden method `display()` in class `B`, the `throws Exception` clause is omitted, which is valid because a subclass can throw fewer exceptions or none at all.

---

2. Which is true about constructors in Java?

A. Constructors can be `final`.
B. Constructors can be overloaded.
C. Constructors can return a value.
D. Constructors can be inherited.

Answer: B
Explanation:
Constructors can be overloaded, but they cannot be inherited, made `final`, or return a value.

---

3. Can a `finally` block execute if a `System.exit()` is called in a `try` block?

A. Yes
B. No
C. Only if it is a checked exception.
D. Only in Java 9+.

Answer: B
Explanation:
When `System.exit()` is called, the JVM terminates immediately, so the `finally` block does not execute.

---

4. What is the output of this code?

```
java


  class Parent {
      int value = 100;

      Parent() {
          System.out.println(value);
          show();
      }

      void show() {
          System.out.println("Parent Show");
      }
  }

  class Child extends Parent {
      int value = 200;

      Child() {
          System.out.println(value);
      }

      @Override
```

```
        void show() {
            System.out.println("Child Show");
        }
    }

    public class Test {
        public static void main(String[] args) {
            new Child();
        }
    }
```

A. 100, Parent Show, 200
B. 100, Child Show, 200
C. 200, Child Show, 200
D. Compilation error

Answer: B
Explanation:
During the `Parent` constructor execution, the overridden `show()` method in `Child` is called, but `value` still refers to the parent's version (`100`) at that time.

---

5. Which of the following allows multiple inheritance in Java?

A. Classes
B. Interfaces
C. Abstract classes
D. None of the above

Answer: B
Explanation:
Multiple inheritance is not supported by classes but is allowed through interfaces.

---

6. What is the output of this code?

```
java


  class Test {
      static {
          System.out.println("Static Block");
      }

      public static void main(String[] args) {
          System.out.println("Main Method");
      }
  }
```

A. Static Block
B. Static Block, Main Method
C. Main Method
D. Compilation error

Answer: B
Explanation:
The static block is executed first when the class is loaded, followed by the `main` method.

7. Which of the following is a valid statement?

A. Abstract methods can have a body.
B. Abstract classes can have constructors.
C. An abstract class must have at least one abstract method.
D. You can create an object of an abstract class.

Answer: B
Explanation:
Abstract classes can have constructors, but you cannot instantiate them directly.

---

8. What happens when a thread is started with `start()` method without overriding the `run()` method?

A. It throws an error.
B. It terminates immediately.
C. The default `run()` method of `Thread` is executed.
D. The thread does not start.

Answer: C
Explanation:
If `run()` is not overridden, the default `run()` method in `Thread` executes, which does nothing.

---

9. What is the purpose of `native` keyword in Java?

A. It defines a static variable.
B. It is used to access OS-level resources.
C. It makes a class unmodifiable.
D. It is used for multi-threading.

Answer: B
Explanation:
The `native` keyword is used to indicate methods implemented in platform-specific native code (e.g., C/C++).

---

10. Which statement about polymorphism is true?

A. It is achieved only through method overloading.
B. It is achieved only through method overriding.
C. It is achieved through both overloading and overriding.
D. It cannot be implemented in Java.

Answer: C
Explanation:
Polymorphism in Java is achieved through method overloading (compile-time) and method overriding (runtime).

---

11. What happens if we do not explicitly call `super()` in a subclass constructor?

A. Compilation error.
B. The JVM automatically calls `super()`.
C. The constructor fails to execute.
D. The superclass is not initialized.

Answer: B
Explanation:
If `super()` is not explicitly called, the compiler automatically inserts it to call the no-arg constructor of the parent class.

---

12. Which statement is true for the `hashCode()` method?

A. Two objects with the same hash code are always equal.
B. It is used to compare objects directly.
C. It is used in hashing-based collections like `HashMap`.
D. It cannot be overridden.

Answer: C
Explanation:
The `hashCode()` method is used in hashing-based collections to improve performance.

---

13. What happens when a `static` method is overridden?

A. It is treated as method overriding.
B. It is treated as method hiding.
C. Compilation error.
D. The child method always executes.

Answer: B
Explanation:
Overriding a static method results in method hiding, where the method is resolved based on the reference type.

---

14. Which keyword prevents a variable from being serialized?

A. final
B. transient
C. static
D. volatile

Answer: B
Explanation:
The `transient` keyword prevents a variable from being included during serialization.

---

15. What happens if a `try` block has a `return` statement but a `finally` block also modifies the return value?

A. The original value is returned.
B. The modified value is returned.

C. Compilation error.
D. Runtime error.

Answer: B
Explanation:
The `finally` block executes after the `try` block, and any changes made to the return value in `finally` are reflected.

---

16. Can we access a `private` constructor outside its class?

A. No
B. Yes, using reflection.
C. Yes, using inheritance.
D. Yes, if the class is in the same package.

Answer: B
Explanation:
A `private` constructor can be accessed outside the class using reflection.

---

17. What happens if a class has multiple `main()` methods with different arguments?

A. Only one is executed.
B. The JVM executes all of them.
C. It causes ambiguity.
D. Compilation error.

Answer: A
Explanation:
The JVM only executes the `main(String[] args)` method.

---

18. Which method is not part of the `Object` class?

A. clone()
B. toString()
C. wait()
D. notifyAll(int)

Answer: D
Explanation:
`notifyAll()` is a valid method, but `notifyAll(int)` does not exist.

---

19. What is the output of this code?

```java


  interface A {
      int value = 10;
  }
```

```
class B implements A {
    void display() {
        value = 20;
        System.out.println(value);
    }
}
```

A. 20
B. 10
C. Compilation error
D. Runtime error

Answer: C
Explanation:
Interface variables are `public static final` by default, so they cannot be modified.

---

20. What happens when a thread calls `sleep()`?

A. The thread releases all locks.
B. The thread releases CPU but retains locks.
C. The thread terminates.
D. The thread continues to execute.

Answer: B
Explanation:
The `sleep()` method pauses the thread and releases the CPU but retains any locks held by the thread.

Let me know if you'd like further refinements or explanations!

give the another 20 mcqs.

Here are **20 more extreme-level Java OOP MCQs** with explanations:

---

1. What is the output of the following code?

```
java


class Parent {
    static void staticMethod() {
        System.out.println("Parent static");
    }
}

class Child extends Parent {
    static void staticMethod() {
        System.out.println("Child static");
    }
}

public class Test {
```

```
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.staticMethod();
    }
}
```

A. Parent static
B. Child static
C. Compilation error
D. Runtime error

Answer: A
Explanation:
Static methods are resolved at compile time based on the reference type (`Parent` in this case). Hence, `Parent.staticMethod()` is called.

---

2. Which modifier is not allowed for a method in an interface (Java 8+)?

A. static
B. default
C. protected
D. private

Answer: C
Explanation:
Methods in an interface cannot be `protected`. They can be `default`, `static`, or `private` in Java 8+.

---

3. What happens if you define a constructor in an `enum`?

A. Compilation error
B. Allowed but cannot call explicitly
C. Allowed and can be called explicitly
D. Runtime error

Answer: B
Explanation:
Enums can have constructors, but they are private and cannot be called explicitly. They are called internally when defining the enum constants.

---

4. What is the output of the following code?

```
java


class Test {
    public static void main(String[] args) {
        System.out.println(10 + 20 + "Java" + 30 + 40);
    }
}
```

A. 30Java3040
B. 30Java70
C. Java3040
D. Compilation error

Answer: A
Explanation:
The expression is evaluated left to right. First, `10 + 20 = 30`, and then `"30" + "Java"` results in `30Java`. The remaining numbers are concatenated as strings.

---

5. Can an anonymous class have a constructor?

A. Yes
B. No
C. Only if it extends another class
D. Only if it implements an interface

Answer: B
Explanation:
Anonymous classes cannot define constructors as they do not have a name.

---

6. What is true about a class with `abstract` methods?

A. It must be declared `final`.
B. It must implement all abstract methods in its parent class.
C. It cannot have constructors.
D. It must be declared `abstract`.

Answer: D
Explanation:
A class containing abstract methods must itself be declared `abstract`.

---

7. Which of the following is invalid?

A. `abstract final class Test {}`
B. `abstract class Test {}`
C. `final class Test {}`
D. `abstract class Test { void method() {} }`

Answer: A
Explanation:
A class cannot be both `abstract` and `final` because `final` prevents inheritance while `abstract` requires it.

---

8. What happens when a `synchronized` method is called by multiple threads?

A. All threads execute simultaneously.
B. Threads wait for the method to finish for one thread.

C. Only `static` methods block access.
D. It causes a deadlock.

Answer: B
Explanation:
When a method is `synchronized`, threads execute it one at a time by locking the object or class.

---

9. What is the output of this code?

```
java


  class Parent {
      private void method() {
          System.out.println("Parent");
      }
  }

  class Child extends Parent {
      private void method() {
          System.out.println("Child");
      }
  }

  public class Test {
      public static void main(String[] args) {
          Parent obj = new Child();
          // obj.method();
      }
  }
```

A. Parent
B. Child
C. Compilation error
D. Runtime error

Answer: C
Explanation:
Private methods are not inherited, so the `method()` call will result in a compilation error as `Parent` does not have a visible `method()`.

---

10. Which is correct about `clone()` in Java?

A. It is a static method.
B. It is a native method.
C. It is available in all classes by default.
D. It must be overridden for cloning to work.

Answer: B
Explanation:
The `clone()` method is a `native` method defined in the `Object` class.

---

11. What happens when you define `hashCode()` without defining `equals()`?

A. It causes a compilation error.
B. It has no effect on object comparisons.
C. It violates the `hashCode()`-`equals()` contract.
D. It automatically generates `equals()`.

Answer: C
Explanation:
If `equals()` is not overridden, the `hashCode()`-`equals()` contract can be violated, leading to incorrect behavior in collections like `HashMap`.

---

12. Which of the following is not a valid way to create a thread in Java?

A. Extending `Thread`
B. Implementing `Runnable`
C. Implementing `Callable`
D. Creating a `Runnable` instance directly

Answer: D
Explanation:
A `Runnable` instance must be passed to a `Thread` to start a thread.

---

13. Which keyword allows a subclass to access its parent class's private members?

A. super
B. this
C. protected
D. None

Answer: D
Explanation:
Private members cannot be directly accessed by a subclass.

---

14. What happens when you declare a `final` method in an abstract class?

A. Compilation error
B. The method cannot be overridden in subclasses.
C. Abstract classes cannot have `final` methods.
D. The `final` modifier is ignored.

Answer: B
Explanation:
A `final` method in an abstract class cannot be overridden but can still exist.

---

15. What is the purpose of `strictfp` in Java?

A. To enforce platform-independent floating-point precision.
B. To make a class immutable.
C. To handle file operations.
D. To optimize performance.

Answer: A
Explanation:
The `strictfp` keyword ensures consistent floating-point calculations across platforms.

---

16. Which exception is thrown when an object is serialized but does not implement `Serializable`?

A. IllegalArgumentException
B. IOException
C. NotSerializableException
D. SerializationException

Answer: C
Explanation:
A `NotSerializableException` is thrown when an object does not implement the `Serializable` interface.

---

17. Which of these is thread-safe?

A. `StringBuilder`
B. `StringBuffer`
C. `HashMap`
D. `ArrayList`

Answer: B
Explanation:
`StringBuffer` is thread-safe as its methods are synchronized.

---

18. What is the purpose of the `volatile` keyword?

A. To ensure atomicity.
B. To guarantee visibility of changes to variables across threads.
C. To make a variable immutable.
D. To enforce thread synchronization.

Answer: B
Explanation:
`volatile` ensures visibility of variable changes across threads but does not guarantee atomicity.

---

19. What is the output of this code?

```
java


  class A {
      static {
          System.out.println("Static A");
      }
      {
          System.out.println("Instance A");
      }
  }

  class B extends A {
```

```
        static {
            System.out.println("Static B");
        }
        {
            System.out.println("Instance B");
        }
    }

    public class Test {
        public static void main(String[] args) {
            B b = new B();
        }
    }
```

A. Static A, Static B, Instance A, Instance B
B. Static A, Instance A, Static B, Instance B
C. Static A, Instance B, Static B, Instance A
D. Compilation error

Answer: A
Explanation:
Static blocks are executed first when the class is loaded, followed by instance blocks during object creation.

---

20. What is the purpose of `@FunctionalInterface`?

A. It forces an interface to have only one abstract method.
B. It allows an interface to have multiple methods.
C. It enforces inheritance.
D. It is used for serialization.

Answer: A
Explanation:
`@FunctionalInterface` ensures that the interface has exactly one abstract method to support functional programming.

Let me know if you'd like more or need clarifications!